



Plan de Pruebas de Automatización

Aplicación: Demoblaze

Tipo de prueba: End-to-End (E2E)

Framework: Serenity BDD + Cucumber + Screenplay + Java



1. Objetivo

Validar el flujo completo de compra en la aplicación, asegurando que un usuario puede:

- Agregar productos al carrito
 - Visualizar el carrito
 - Completar el formulario
 - Finalizar la compra exitosamente
-



2. Alcance

✓ Incluye

- Flujo E2E completo de compra
- Validación de alertas ("Product added")
- Validación de productos en carrito
- Validación de mensaje de compra exitosa

✗ No incluye

- Pruebas de rendimiento
 - Pruebas de seguridad
 - Pruebas cross-browser (opcional)
-



3. Escenario principal (E2E)

● Escenario: Compra exitosa de productos

Dado que el usuario ingresa a la página

Cuando agrega dos productos al carrito

Y navega al carrito

Y completa el formulario de compra
Entonces finaliza la compra exitosamente

4. Casos de prueba

CP01 - Agregar producto al carrito

- Seleccionar producto
 - Click en "Add to cart"
 - Validar alert: "Product added"
-

CP02 - Agregar segundo producto

- Repetir flujo anterior
 - Validar alert nuevamente
-

CP03 - Visualizar carrito

- Ir a "Cart"
 - Validar que existan 2 productos
-

CP04 - Completar formulario

- Click en "Place Order"
 - Llenar campos:
 - Name
 - Country
 - City
 - Card
 - Month
 - Year
-

CP05 - Finalizar compra

- Click en "Purchase"
- Validar mensaje de éxito:
 - "Thank you for your purchase!"

5. Arquitectura (Screenplay)

Actores

- Usuario comprador

Tasks

- Abrir la página
- Seleccionar producto
- Agregar producto al carrito
- Navegar al carrito
- Completar formulario
- Confirmar compra

Questions

- Validar texto del alert
- Validar cantidad de productos
- Validar mensaje final

6. Herramientas

- Java
- Gradle
- Serenity BDD
- Cucumber
- Screenplay
- Selenium WebDriver
- WebDriverManager

7. Evidencias

- Reporte de ejecución (Serenity Reports)
- Logs de ejecución
- Capturas automáticas en cada paso

8. Entregables requeridos

- Código fuente en GitHub
 - `README.txt` (pasos de ejecución)
 - `conclusiones.txt` (hallazgos)
 - Reportes generados (`target/site/serenity`)
-

9. Riesgos

- Dependencia de alertas JS (pueden fallar si no se manejan bien)
 - Elementos dinámicos (necesitan esperas explícitas)
 - Datos quemados en formulario
-

10. Buenas prácticas aplicadas

- Uso de Screenplay Pattern
- Esperas explícitas (`WaitUntil`)
- Evitar `Thread.sleep`
- Separación de responsabilidades (Tasks, Questions, Targets)